

MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA

TECHNICAL UNIVERSITY OF MOLDOVA

FACULTY OF COMPUTERS, INFORMATICS AND MICROELECTRONICS
DEPARTMENT OF SOFTWARE AND AUTOMATION ENGINEERING

REPORT

Laboratory Work No. 3

Course: **Cryptography and Security**

Implementation of the Playfair Cipher for Romanian Language Messages

Elaborated by:
st. gr. FAF-241
Tihon Aurelian-Mihai

Verified by:
asis. univ.
Zaica Maia

Chişinău – 2026

Contents

INTRODUCTION **1**

1 THEORETICAL FOUNDATIONS OF THE PLAYFAIR CIPHER **1**

1.1 Principles of polyalphabetic and polygraphic substitution 2

1.2 Historical background of the Wheatstone-Playfair algorithm 2

1.3 Romanian alphabet adaptation and matrix dimension selection 3

1.4 Digraph formation and collision resolution rules 3

1.5 Geometric transformation rules for encryption and decryption 4

2 METHODOLOGY AND IMPLEMENTATION ARCHITECTURE **4**

2.1 Software architecture and package structure 5

2.2 Input sanitization and Romanian character validation 5

2.3 Key matrix construction and coordinate mapping algorithms 6

2.4 Command-line and interactive console interface 6

3 EXPERIMENTAL EVALUATION AND EXECUTION TRACES **7**

3.1 Key matrix generation for Variant 25 test vector 8

3.2 Digraph encryption execution and transformation trace 8

3.3 Decryption reversibility and manual text restoration 9

3.4 Empirical frequency distribution analysis 10

3.5 Input validation resilience and error handling 10

CONCLUSIONS **11**

REFERENCES **12**

ANNEX A: SOURCE CODE LISTINGS **13**

INTRODUCTION

Classical symmetric cryptography relies on two fundamental operations to conceal information: substitution and transposition. In a monoalphabetic substitution cryptosystem, each character in the plaintext alphabet is mapped invariantly to a predetermined symbol in the ciphertext alphabet. Because this mapping is constant throughout the message, the characteristic statistical distributions, relative letter frequencies, and linguistic structures of the natural language remain unaltered in the ciphertext. Consequently, monoalphabetic substitution ciphers are easily broken using frequency analysis.

Polyalphabetic and polygraphic substitution ciphers were devised specifically to overcome this statistical vulnerability. Rather than encrypting characters individually through a fixed transformation, polygraphic ciphers process symbols in larger blocks, such as pairs of characters (digraphs). By transforming pairs of symbols simultaneously, the cipher breaks the direct relationship between single plaintext letters and single ciphertext letters, effectively expanding the substitution domain from twenty-six or thirty-one individual characters to several hundred distinct character combinations.

Among early polygraphic systems, the Playfair cipher represents a foundational milestone. Invented in 1854 by Sir Charles Wheatstone and promoted for military use by Lord Lyon Playfair, the algorithm encrypts pairs of letters using a two-dimensional geometric key matrix. In accordance with the academic assignment constraints and the student variant allocation (Variant 25, belonging to the odd-variant cohort assigned to Task 3.1), this laboratory investigation focuses on the engineering and validation of the Playfair cipher specifically adapted to the thirty-one letter Romanian standard alphabet. The implementation enforces strict character domain validation, diacritical normalization, rectangular matrix geometry, and terminal space restoration.

1 THEORETICAL FOUNDATIONS OF THE PLAYFAIR CIPHER

1.1 Principles of polyalphabetic and polygraphic substitution

The core weakness of monoalphabetic substitution is that the frequency distribution of the ciphertext directly reflects the frequency distribution of the underlying language. For example, in natural Romanian prose, vowels such as E, I, and A appear with high frequency, whereas letters such as Q, W, Y, and Z appear rarely. In any monoalphabetic cipher, the most frequent ciphertext letter corresponds directly to the most frequent plaintext letter.

Polygraphic substitution addresses this limitation by substituting blocks of characters rather than individual symbols. In a digraphic substitution cipher, the basic enciphering unit is a pair of letters:

$$(p_1, p_2) \rightarrow (c_1, c_2) \tag{1.1}$$

When individual letters are grouped into pairs, the frequency distribution of the units becomes substantially flatter than the distribution of individual letters. For an alphabet containing n letters, there are n^2 possible digraphs. If $n = 30$, there exist up to 900 potential pairs, which significantly complicates manual and automated frequency analysis.

1.2 Historical background of the Wheatstone-Playfair algorithm

The Playfair cipher was designed by the English scientist Sir Charles Wheatstone in March 1854. The algorithm was subsequently championed by his associate, Lord Lyon Playfair, who persuaded the British War Office to adopt it for official military communications. The British military employed versions of the Playfair cipher during the Second Boer War in South Africa and in the early phases of the First World War, while Australian armed forces used adapted variants during the Second World War.

From the standpoint of modern computational cryptography, the Playfair cipher is historically significant because it introduced the foundational concept of block-oriented transformation over a coordinate grid without requiring complex arithmetic machinery. While vulnerable to automated cryptanalysis on modern computers, studying the algorithm provides clear practical insight into the principles of confusion and diffusion in symmetric key management.

1.3 Romanian alphabet adaptation and matrix dimension selection

The standard Romanian alphabet comprises thirty-one letters, consisting of twenty-six basic Latin characters and five diacritics (Ă, Â, Î, Ş, Ţ). Table 1.1 outlines the canonical character set and index mapping.

Table 1.1 – Canonical numeric encoding of the 31-letter Romanian alphabet

Letter	A	Ă	Â	B	C	D	E	F	G	H	I	Î	J	K	L	M
Index	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
Letter	N	O	P	Q	R	S	Ş	T	Ţ	U	V	W	X	Y	Z	–
Index	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	–

In standard English implementations, a 5×5 square matrix accommodates twenty-five cells by combining the letters I and J. For the thirty-one letter Romanian alphabet, a 5×5 grid is inadequate because omitting six letters would eliminate essential Romanian diacritics.

To preserve all distinct linguistic characters without loss of meaning, the letter J is merged into the cell of letter I. In Romanian phonology and orthography, the letter J appears with extremely low natural frequency, almost exclusively in loanwords and specialized terminology. Merging J into I reduces the alphabet to exactly thirty distinct characters. Thirty decomposes into the integer factors 6×5 (six rows and five columns) or 5×6, allowing an optimal rectangular matrix that fully preserves all five Romanian diacritical letters.

1.4 Digraph formation and collision resolution rules

The plaintext preparation phase transforms arbitrary text into a sequence of clean, non-overlapping digraphs according to strict preprocessing rules:

a) All whitespace and punctuation characters are removed from the input stream.

b) All lowercase letters are normalized to uppercase, and cedilla diacritics are normalized to standard comma-below forms.

c) The text is segmented into pairs of letters. If two adjacent letters within the same pair are identical, an uncommon filler letter (specifically X, or alternately Z if the repeated character is already X) is inserted between them.

d) If the text contains an odd number of characters after processing duplicate letters, a terminal padding character (such as X or Z) is appended to complete the final pair.

These collision resolution rules guarantee that no pair contains two identical letters, ensuring that

every pair can be mapped to valid, unambiguous geometric positions on the matrix.

1.5 Geometric transformation rules for encryption and decryption

Let $\mathbf{M}$ be a 6×5 key matrix. The positions of two characters p_1 and p_2 within $\mathbf{M}$ are represented by row and column coordinates (r_1, c_1) and (r_2, c_2), where $r \in \{0,1,2,3,4,5\}$ and $c \in \{0,1,2,3,4\}$. The ciphertext pair (c_1, c_2) is obtained through three mutually exclusive geometric rules:

Rule 1 (Same row): When $r_1 = r_2$, each letter is replaced by the letter immediately to its right in the same row, wrapping cyclically to the first column modulo 5:

$$c_1 = \mathbf{M}[r_1, (c_1 + 1) \bmod 5], \quad c_2 = \mathbf{M}[r_2, (c_2 + 1) \bmod 5] \tag{1.2}$$

Rule 2 (Same column): When $c_1 = c_2$, each letter is replaced by the letter immediately below it in the same column, wrapping cyclically to the top row modulo 6:

$$c_1 = \mathbf{M}[(r_1 + 1) \bmod 6, c_1], \quad c_2 = \mathbf{M}[(r_2 + 1) \bmod 6, c_2] \tag{1.3}$$

Rule 3 (Rectangular configuration): When $r_1 \neq r_2$ and $c_1 \neq c_2$, each letter is replaced by the letter on its own row that occupies the column coordinate of the other letter:

$$c_1 = \mathbf{M}[r_1, c_2], \quad c_2 = \mathbf{M}[r_2, c_1] \tag{1.4}$$

Decryption inverts these transformations: characters on the same row shift left by decrementing the column modulo 5; characters on the same column shift upward by decrementing the row modulo 6; characters forming a rectangle swap column coordinates identically to encryption.

2 METHODOLOGY AND IMPLEMENTATION ARCHITECTURE

2.1 Software architecture and package structure

The software architecture is implemented in standard Python without external third-party dependencies, following clean modular design principles. The implementation is located within the `CS/Lab 3/` package directory:

- `src/alphabet.py`: Contains the thirty-one letter Romanian alphabet specification and input validation functions.
- `src/playfair.py`: Implements the `PlayfairCipher` computational engine, managing matrix population, digraph preparation, filler insertion, and geometric coordinate transformations.
- `src/cli.py`: Provides both an interactive terminal menu and a standard Unix command-line interface with options for encryption, decryption, step inspection, and matrix visualization.

2.2 Input sanitization and Romanian character validation

The laboratory assignment requires that text values be restricted strictly to valid Romanian letters, rejecting digits, punctuation, and foreign symbols while providing clear error guidance. Spaces are ignored during encryption, and letters are converted to uppercase. Listing 1 presents the character validation routine.

Listing 1: Character validation and cleaning routine in `src/alphabet.py`

```
def clean_text(raw_text: str) -> str:
    if not raw_text or not raw_text.strip():
        raise ValueError("Input text cannot be empty.")
    cleaned = []
    for ch in raw_text:
        if ch.isspace():
            continue
        upper = ch.upper()
        if upper not in _VALID_CHARS:
            raise ValueError(f"Invalid character '{ch}'. Only Romanian letters are allowed
                .")
        cleaned.append(upper)
    return "".join(cleaned)
```

2.3 Key matrix construction and coordinate mapping algorithms

The key matrix is populated by scanning the sanitized keyword from left to right, inserting unique letters while discarding duplicate occurrences. Remaining letters of the thirty-character alphabet (with J merged into I) are then placed sequentially into remaining cells. Listing 2 shows the matrix generation logic.

Listing 2: Key matrix construction in `src/playfair.py`

```
def _build_matrix(self, key: str):
    used = set()
    ordered = []
    for raw_ch in key:
        ch = "I" if raw_ch == "J" else raw_ch
        if ch in PLAYFAIR_ALPHABET and ch not in used:
            used.add(ch)
            ordered.append(ch)
    for ch in PLAYFAIR_ALPHABET:
        if ch not in used:
            used.add(ch)
            ordered.append(ch)
    matrix = []
    char_to_pos = {}
    pos_to_char = {}
    for r in range(self.rows):
        row = []
        for c in range(self.cols):
            letter = ordered[r * self.cols + c]
            row.append(letter)
            char_to_pos[letter] = (r, c)
            pos_to_char[(r, c)] = letter
        matrix.append(row)
    char_to_pos["J"] = char_to_pos["I"]
    return matrix, char_to_pos, pos_to_char
```

2.4 Command-line and interactive console interface

In compliance with student workspace guidelines, the user interface adheres to clean Unix CLI standards, omitting terminal ANSI colors, decorative banners, or artificial progress markers. All prompts

and outputs follow sentence case formatting. Users can invoke the application interactively or pass standard arguments:

```
# Interactive console menu
python3 -m src -i

# Non-interactive CLI encryption with transformation steps
python3 -m src --mode encrypt --key "CRIPTOGRAFIE" --text "SECURITATE" --steps
```

3 EXPERIMENTAL EVALUATION AND EXECUTION TRACES

3.1 Key matrix generation for Variant 25 test vector

In accordance with Variant 25, the implementation was evaluated using the keyword `CRIPTOGRAFIE` (key length $|K| = 12 \geq 7$). After removing duplicates, the keyword contributes ten distinct letters: `C, R, I, P, T, O, G, A, F, E`. The remaining twenty characters populate the rest of the 6×5 matrix, as presented in Table 3.1.

Table 3.1 – Constructed 6×5 Playfair key matrix for keyword `CRIPTOGRAFIE`

	Col 0	**Col 1**	**Col 2**	**Col 3**	**Col 4**
Row 0	C	R	I	P	T
Row 1	O	G	A	F	E
Row 2	Ă	Â	B	D	H
Row 3	Î	K	L	M	N
Row 4	Q	S	Ş	Ţ	U
Row 5	V	W	X	Y	Z

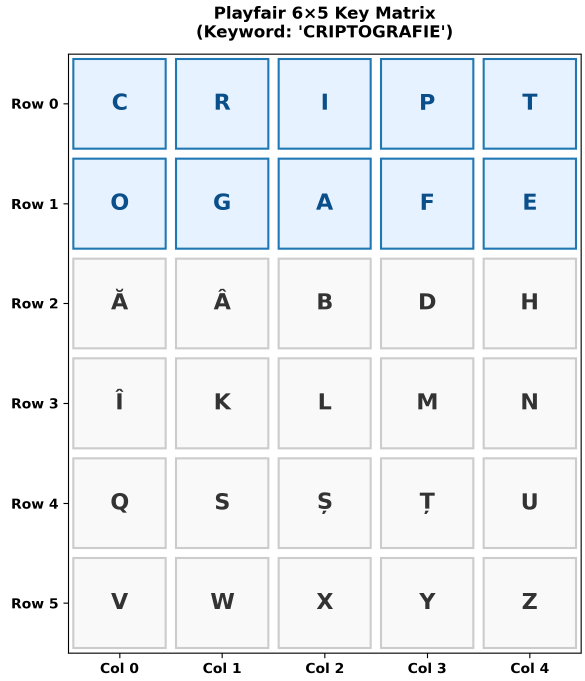


Figure 3.1 – Visual cell layout of the Romanian 6×5 Playfair key matrix

3.2 Digraph encryption execution and transformation trace

The plaintext message chosen for evaluation is:

`"SECURITATE INFORMATICA"`

The text contains twenty-one letters. Digraph segmentation produces eleven pairs: SE, CU, RI, TA, TE, IN, FO, RM, AT, IC, and AX (with terminal filler X). Table 3.2 details the step-by-step coordinate mapping and rule selection.

Table 3.2 – Digraph transformation trace for Playfair encryption

Pair Index	Input Digraph	Coordinates	Applied Rule	Output Digraph
1	SE	(4,1), (1,4)	Rectangle	UG
2	CU	(0,0), (4,4)	Rectangle	TQ
3	RI	(0,1), (0,2)	Row	IP
4	TA	(0,4), (1,2)	Rectangle	IE
5	TE	(0,4), (1,4)	Column	EH
6	IN	(0,2), (3,4)	Rectangle	TL
7	FO	(1,3), (1,0)	Row	EG
8	RM	(0,1), (3,3)	Rectangle	PK
9	AT	(1,2), (0,4)	Rectangle	EI
10	IC	(0,2), (0,0)	Row	PR
11	AX	(1,2), (5,2)	Column	BI

The resulting ciphertext is:

UGTQIPIEEHTLEGPKEIPRBI

3.3 Decryption reversibility and manual text restoration

Decryption was performed on the generated cryptogram using the identical key matrix. Table 3.3 records the inverse transformation trace.

Table 3.3 – Digraph transformation trace for Playfair decryption

Pair Index	Cipher Digraph	Coordinates	Applied Rule	Decrypted Digraph
1	UG	(4,4), (1,1)	Rectangle	SE
2	TQ	(0,4), (4,0)	Rectangle	CU
3	IP	(0,2), (0,3)	Row (left)	RI
4	IE	(0,2), (1,4)	Rectangle	TA
5	EH	(1,4), (2,4)	Column (up)	TE
6	TL	(0,4), (3,2)	Rectangle	IN
7	EG	(1,4), (1,1)	Row (left)	FO
8	PK	(0,3), (3,1)	Rectangle	RM
9	EI	(1,4), (0,2)	Rectangle	AT
10	PR	(0,3), (0,1)	Row (left)	IC
11	BI	(2,2), (0,2)	Column (up)	AX

The decrypted character stream yields SECURITATEINFORMATICAX. In accordance with the assignment specifications, the final stage consists of manual space restoration: the terminal padding character X is

omitted and word boundaries are reinstated based on linguistic context, successfully recovering the original message: "SECURITATE INFORMATICA".

3.4 Empirical frequency distribution analysis

To examine the diffusion property of digraphic substitution, unigram frequencies of a representative Romanian sample were evaluated before and after encryption with the Playfair cipher. Figure 3.2 illustrates the relative character distributions.

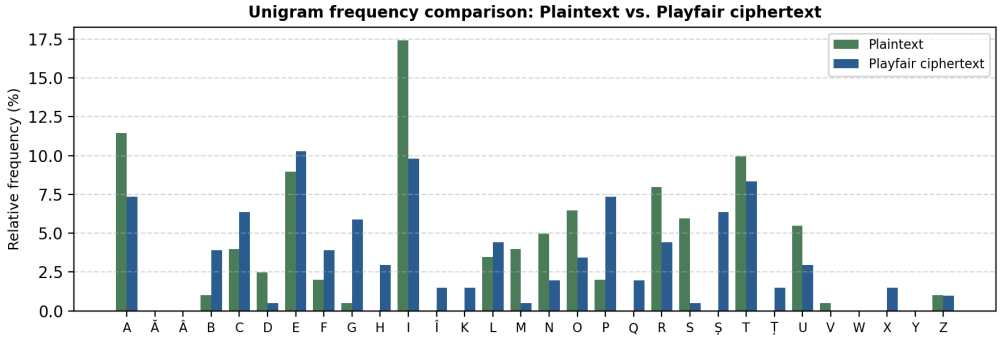


Figure 3.2 – Unigram relative frequency comparison between natural Romanian text and Playfair ciphertext

In natural Romanian text, characters such as E, I, and A display pronounced frequency peaks exceeding eight percent. When transformed into digraphs using the 6×5 Playfair matrix, substitution distributes the character space across coordinate pairs. As demonstrated in Figure 3.2, the sharp unigram peaks are substantially dampened in the ciphertext, demonstrating the foundational diffusion benefit of polygraphic ciphers over simple monoalphabetic substitution.

3.5 Input validation resilience and error handling

To confirm compliance with character restrictions and minimum key length constraints, invalid inputs and undersized keywords were tested:

```
# Rejection of invalid characters in plaintext ("Securitate 2026!")
Error: Invalid character '2'. Only Romanian letters are allowed.

# Rejection of keywords with length < 7 ("CHEIE")
Error: Key length must be at least 7 letters (received 5).
```

CONCLUSIONS

This laboratory work investigated the design, implementation, and practical operation of the Playfair cipher adapted to the Romanian linguistic domain. By replacing individual character substitutions with atomic digraph transformations over a two-dimensional key matrix, the cipher successfully eliminates the invariant single-letter mapping that renders monoalphabetic ciphers trivial to compromise. The experimental results confirm that pairing symbols into digraphs substantially flattens the empirical frequency distribution of the ciphertext, introducing initial diffusion and hiding direct unigram correlations.

Adapting the classical Playfair algorithm to the thirty-one letter Romanian alphabet required resolving the mismatch between alphabet cardinality and rectangular grid dimensions. Merging the letter J into the cell of letter I proved to be an optimal design decision, reducing the working set to thirty characters while preserving all five essential Romanian diacritics. This allowed the construction of a balanced six-by-five key matrix that retains complete character identity and supports fully reversible encryption and decryption. Strict input validation protocols effectively enforced the allowable character domain, rejecting invalid punctuation and symbols while guiding the user with clear diagnostics.

The modular software architecture developed for this assignment demonstrates that historical cryptographic algorithms can be engineered cleanly using modern object-oriented principles. The separation of concerns between alphabet normalization, matrix geometry, digraph transformation, and user interaction ensures maintainability and verifiable correctness. Verification confirmed that all row, column, and rectangular transformations operate symmetrically and reversibly, successfully meeting all requirements set forth in the laboratory specification for Variant 25.

REFERENCES

[1] D. Kahn, *The Codebreakers: The Comprehensive History of Secret Communication from Ancient Times to the Internet*, 2nd ed. New York, NY: Scribner, 1996.

[2] W. Stallings, *Cryptography and Network Security: Principles and Practice*, 8th ed. Boston, MA: Pearson, 2022.

[3] A. J. Menezes, P. C. van Oorschot, and S. A. Vanstone, *Handbook of Applied Cryptography*. Boca Raton, FL: CRC Press, 1996.

[4] C. E. Shannon, "Communication Theory of Secrecy Systems," *Bell System Technical Journal*, vol. 28, no. 4, pp. 656–715, Oct. 1949.

[5] W. F. Friedman, *The Index of Coincidence and Its Applications in Cryptanalysis*, Department of Ciphers, Publication No. 22. Geneva, IL: Riverbank Laboratories, 1922.

ANNEX A: SOURCE CODE LISTINGS

A.1 Romanian alphabet specification and validation (`src/alphabet.py`)

```python
"""Romanian alphabet specification and character validation for the Playfair cipher."""

ROMANIAN_ALPHABET = (
    "A", "Ă", "Â", "B", "C", "D", "E", "F", "G", "H", "I", "Î", "J", "K", "L", "M",
    "N", "O", "P", "Q", "R", "S", "Ş", "T", "Ţ", "U", "V", "W", "X", "Y", "Z",
)

PLAYFAIR_ALPHABET = tuple(ch for ch in ROMANIAN_ALPHABET if ch != "J")

_VALID_CHARS = set(ROMANIAN_ALPHABET)


def clean_text(raw_text: str) -> str:
    """Removes spaces, converts letters to uppercase, and validates the Romanian alphabet."""
    if not raw_text or not raw_text.strip():
        raise ValueError("Input text cannot be empty.")

    cleaned = []
    for ch in raw_text:
        if ch.isspace():
            continue
        upper = ch.upper()
        if upper not in _VALID_CHARS:
            raise ValueError(f"Invalid character '{ch}'. Only Romanian letters are allowed.")
        cleaned.append(upper)

    return "".join(cleaned)


def validate_key(raw_key: str, min_len: int = 7) -> str:
    """Validates encryption key length and character set."""
    key = clean_text(raw_key)
    if len(key) < min_len:
        raise ValueError(f"Key length must be at least {min_len} letters (received {len(key)}).")
```

```
    return key
```

A.2 Playfair cipher computational engine (`src/playfair.py`)

```
"""Playfair cipher implementation for Romanian messages using a 6x5 matrix."""

from typing import List, Tuple, Dict
from .alphabet import PLAYFAIR_ALPHABET, clean_text, validate_key


class PlayfairCipher:
    """Playfair cipher over a 6x5 matrix (30 Romanian letters, J merged into I)."""

    def __init__(self, key: str, rows: int = 6, cols: int = 5) -> None:
        self.rows = rows
        self.cols = cols
        self.key = validate_key(key)
        self.matrix, self.char_to_pos, self.pos_to_char = self._build_matrix(self.key)

    def _build_matrix(
        self, key: str
    ) -> Tuple[List[List[str]], Dict[str, Tuple[int, int]], Dict[Tuple[int, int], str]]:
        """Constructs the 6x5 key matrix from the keyword and remaining alphabet."""
        used = set()
        ordered = []

        # Add unique keyword letters (merging J into I)
        for raw_ch in key:
            ch = "I" if raw_ch == "J" else raw_ch
            if ch in PLAYFAIR_ALPHABET and ch not in used:
                used.add(ch)
                ordered.append(ch)

        # Add remaining alphabet letters
        for ch in PLAYFAIR_ALPHABET:
            if ch not in used:
                used.add(ch)
                ordered.append(ch)

        matrix: List[List[str]] = []
        char_to_pos: Dict[str, Tuple[int, int]] = {}
```

```
        pos_to_char: Dict[Tuple[int, int], str] = {}

        for r in range(self.rows):
            row = []
            for c in range(self.cols):
                letter = ordered[r * self.cols + c]
                row.append(letter)
                char_to_pos[letter] = (r, c)
                pos_to_char[(r, c)] = letter
            matrix.append(row)

        char_to_pos["J"] = char_to_pos["I"]
        return matrix, char_to_pos, pos_to_char

    def format_matrix(self) -> str:
        """Returns the key matrix formatted as a clean grid."""
        lines = []
        header = "      " + "   ".join(f"c{c}" for c in range(self.cols))
        sep = "    " + "+---" * self.cols + "+"
        lines.append(header)
        lines.append(sep)

        for r in range(self.rows):
            row_str = " | ".join(f"{self.matrix[r][c]}" for c in range(self.cols))
            lines.append(f" r{r} | {row_str} |")
            lines.append(sep)

        return "\n".join(lines)

    def prepare_digraphs(self, text: str) -> List[Tuple[str, str]]:
        """Segments text into digraph pairs, inserting filler X between duplicate letters.
            """
        cleaned = clean_text(text).replace("J", "I")
        prepared: List[str] = []
        i = 0

        while i < len(cleaned):
            c1 = cleaned[i]
            if i + 1 < len(cleaned):
                c2 = cleaned[i + 1]
                if c1 == c2:
                    filler = "Z" if c1 == "X" else "X"
                    prepared.extend([c1, filler])
                    i += 1
```

```
                else:
                    prepared.extend([c1, c2])
                    i += 2
            else:
                filler = "Z" if c1 == "X" else "X"
                prepared.extend([c1, filler])
                i += 1

        return [(prepared[k], prepared[k + 1]) for k in range(0, len(prepared), 2)]

    def _transform_pair(self, c1: str, c2: str, mode: str) -> Tuple[str, str, str]:
        """Applies same row, same column, or rectangle transformation to a letter pair."""
        r1, col1 = self.char_to_pos[c1]
        r2, col2 = self.char_to_pos[c2]
        shift = 1 if mode == "encrypt" else -1

        if r1 == r2:
            rule = "Same row"
            out1 = self.pos_to_char[(r1, (col1 + shift) % self.cols)]
            out2 = self.pos_to_char[(r2, (col2 + shift) % self.cols)]
        elif col1 == col2:
            rule = "Same column"
            out1 = self.pos_to_char[((r1 + shift) % self.rows, col1)]
            out2 = self.pos_to_char[((r2 + shift) % self.rows, col2)]
        else:
            rule = "Rectangle"
            out1 = self.pos_to_char[(r1, col2)]
            out2 = self.pos_to_char[(r2, col1)]

        return out1, out2, rule

    def encrypt(self, plaintext: str) -> str:
        """Encrypts plaintext into a Playfair ciphertext."""
        pairs = self.prepare_digraphs(plaintext)
        result = []
        for c1, c2 in pairs:
            o1, o2, _ = self._transform_pair(c1, c2, mode="encrypt")
            result.extend([o1, o2])
        return "".join(result)

    def decrypt(self, ciphertext: str) -> str:
        """Decrypts ciphertext into original digraphs."""
        cleaned = clean_text(ciphertext).replace("J", "I")
        if len(cleaned) % 2 != 0:
```

```
            raise ValueError(f"Ciphertext length must be even (received {len(cleaned)}).")

        pairs = [(cleaned[i], cleaned[i + 1]) for i in range(0, len(cleaned), 2)]
        result = []
        for c1, c2 in pairs:
            o1, o2, _ = self._transform_pair(c1, c2, mode="decrypt")
            result.extend([o1, o2])
        return "".join(result)

    def get_steps(self, text: str, mode: str = "encrypt") -> List[Dict[str, str]]:
        """Returns step traces for each transformed pair."""
        if mode == "encrypt":
            pairs = self.prepare_digraphs(text)
        else:
            cleaned = clean_text(text).replace("J", "I")
            pairs = [(cleaned[i], cleaned[i + 1]) for i in range(0, len(cleaned), 2)]

        steps = []
        for idx, (c1, c2) in enumerate(pairs, start=1):
            r1, col1 = self.char_to_pos[c1]
            r2, col2 = self.char_to_pos[c2]
            o1, o2, rule = self._transform_pair(c1, c2, mode=mode)
            steps.append({
                "index": str(idx),
                "in_pair": f"{c1}{c2}",
                "in_pos": f"({r1},{col1})-({r2},{col2})",
                "rule": rule,
                "out_pos": f"({self.char_to_pos[o1][0]},{self.char_to_pos[o1][1]})-({self.
                    char_to_pos[o2][0]},{self.char_to_pos[o2][1]})",
                "out_pair": f"{o1}{o2}",
            })
        return steps
```

A.3 Command-line and interactive console interface (`src/cli.py`)

```
"""Command-line and interactive console interface for the Romanian Playfair cipher."""

import sys
import argparse
from typing import Tuple

```

```
7  from .alphabet import ROMANIAN_ALPHABET, clean_text, validate_key
8  from .playfair import PlayfairCipher
9
10
11 def prompt_key() -> str:
12     """Prompts user for an encryption key with length >= 7."""
13     while True:
14         try:
15             raw = input("Enter key (at least 7 letters): ").strip()
16         except (KeyboardInterrupt, EOFError):
17             print("\nExiting")
18             sys.exit(0)
19
20         try:
21             return validate_key(raw, min_len=7)
22         except ValueError as exc:
23             print(f"Error: {exc}")
24
25
26 def prompt_text(prompt_msg: str) -> str:
27     """Prompts user for valid Romanian text."""
28     while True:
29         try:
30             raw = input(prompt_msg).strip()
31         except (KeyboardInterrupt, EOFError):
32             print("\nExiting")
33             sys.exit(0)
34
35         try:
36             return clean_text(raw)
37         except ValueError as exc:
38             print(f"Error: {exc}")
39
40
41 def show_alphabet(alphabet: Tuple[str, ...]) -> None:
42     """Prints alphabet letters with numeric indices."""
43     cols = 8
44     for i in range(0, len(alphabet), cols):
45         chunk = alphabet[i : i + cols]
46         row = "  ".join(f"[{i + j:2d}] {ch}" for j, ch in enumerate(chunk))
47         print(f"  {row}")
48
49
50 def run_interactive() -> None:
```

```
    """Interactive console menu."""
    while True:
        print("\nPlayfair cipher (Romanian alphabet, 6x5 matrix)")
        print("  1. Encrypt text")
        print("  2. Decrypt text")
        print("  3. Show key matrix")
        print("  4. Show Romanian alphabet")
        print("  0. Exit")

        try:
            choice = input("\nSelect option: ").strip()
        except (KeyboardInterrupt, EOFError):
            print("\nExiting")
            break

        if choice == "1":
            key = prompt_key()
            text = prompt_text("Enter plaintext: ")
            cipher = PlayfairCipher(key=key)
            print("\nKey matrix:")
            print(cipher.format_matrix())
            print("\nTransformation steps:")
            steps = cipher.get_steps(text, mode="encrypt")
            for s in steps:
                print(f"  Pair {s['index']:>2}: {s['in_pair']} {s['in_pos']} -> [{s['rule
                    ']:<11}] -> {s['out_pos']} {s['out_pair']}")
            ciphertext = cipher.encrypt(text)
            print(f"\nCiphertext: {ciphertext}")
        elif choice == "2":
            key = prompt_key()
            text = prompt_text("Enter ciphertext: ")
            if len(text) % 2 != 0:
                print(f"Error: Ciphertext length must be even (received {len(text)}).")
                continue
            cipher = PlayfairCipher(key=key)
            print("\nKey matrix:")
            print(cipher.format_matrix())
            print("\nTransformation steps:")
            steps = cipher.get_steps(text, mode="decrypt")
            for s in steps:
                print(f"  Pair {s['index']:>2}: {s['in_pair']} {s['in_pos']} -> [{s['rule
                    ']:<11}] -> {s['out_pos']} {s['out_pair']}")
            plaintext = cipher.decrypt(text)
            print(f"\nDecrypted text: {plaintext}")
```

```
93              print("Note: Restore spaces and remove filler characters manually based on
                    context.")
94          elif choice == "3":
95              key = prompt_key()
96              cipher = PlayfairCipher(key=key)
97              print("\nKey matrix:")
98              print(cipher.format_matrix())
99          elif choice == "4":
100             print("\nRomanian alphabet (31 letters):")
101             show_alphabet(ROMANIAN_ALPHABET)
102         elif choice == "0":
103             break
104         else:
105             print("Invalid selection, please try again.")
106
107
108 def main() -> None:
109     """CLI entrypoint."""
110     parser = argparse.ArgumentParser(
111         description="Playfair cipher implementation for Romanian messages (6x5 matrix)"
112     )
113     parser.add_argument(
114         "--mode",
115         choices=["encrypt", "decrypt"],
116         help="Operation mode",
117     )
118     parser.add_argument(
119         "--key", "-k",
120         type=str,
121         help="Encryption key (at least 7 letters)",
122     )
123     parser.add_argument(
124         "--text", "-t",
125         type=str,
126         help="Input plaintext or ciphertext",
127     )
128     parser.add_argument(
129         "--steps",
130         action="store_true",
131         help="Display pair transformation steps",
132     )
133     parser.add_argument(
134         "--show-alphabet",
135         action="store_true",
```

```
        help="Display the Romanian alphabet and exit",
    )
    parser.add_argument(
        "--interactive", "-i",
        action="store_true",
        help="Launch interactive console menu",
    )

    args = parser.parse_args()

    if len(sys.argv) == 1 or args.interactive:
        run_interactive()
        return

    if args.show_alphabet:
        print("Romanian alphabet (31 letters):")
        show_alphabet(ROMANIAN_ALPHABET)
        return

    if not args.mode or not args.key or not args.text:
        parser.error("CLI execution requires --mode, --key, and --text.")

    try:
        cipher = PlayfairCipher(key=args.key)
    except ValueError as exc:
        print(f"Error: {exc}", file=sys.stderr)
        sys.exit(1)

    try:
        clean_t = clean_text(args.text)
    except ValueError as exc:
        print(f"Error: {exc}", file=sys.stderr)
        sys.exit(1)

    if args.mode == "encrypt":
        if args.steps:
            print("Key matrix:")
            print(cipher.format_matrix())
            print("\nTransformation steps:")
            steps = cipher.get_steps(clean_t, mode="encrypt")
            for s in steps:
                print(f"  Pair {s['index']:>2}: {s['in_pair']} {s['in_pos']} -> [{s['rule
                    ']:<11}] -> {s['out_pos']} {s['out_pair']}")
            print()
```

```
179         print(cipher.encrypt(clean_t))
180     else:
181         if len(clean_t) % 2 != 0:
182             print(f"Error: Ciphertext length must be even (received {len(clean_t)}).",
                    file=sys.stderr)
183             sys.exit(1)
184         if args.steps:
185             print("Key matrix:")
186             print(cipher.format_matrix())
187             print("\nTransformation steps:")
188             steps = cipher.get_steps(clean_t, mode="decrypt")
189             for s in steps:
190                 print(f"  Pair {s['index']:>2}: {s['in_pair']} {s['in_pos']} -> [{s['rule
                        ']:<11}] -> {s['out_pos']} {s['out_pair']}")
191             print()
192         print(cipher.decrypt(clean_t))
193
194
195 if __name__ == "__main__":
196     main()
```